```
"""
Obsidian Markdown 导出器
将 Gemini 结构化分析结果转为 Obsidian 笔记
"""

import logging
from datetime import datetime
from pathlib import Path
from typing import Optional

logger = logging.getLogger(__name__)


class ObsidianExporter:
    """
    导出为 Obsidian 兼容的 Markdown 笔记

    目录结构:
        vault/
        ├── 个股/
        │   └── 600893-航发动力/
        │       ├── 2024年报分析.md
        │       └── 投资论文.md    ← 双链锚点（需手动维护）
        └── 财报分析模板.md

    使用方法:
        exporter = ObsidianExporter(vault_dir='~/obsidian/fintech')
        exporter.export('600893.SH', 2024, result_dict)
        exporter.batch_export({'600893.SH_2024': result_dict, ...})
    """

    def __init__(self, vault_dir: str = "./obsidian_vault"):
        self.vault_dir = Path(vault_dir).expanduser()
        self.vault_dir.mkdir(parents=True, exist_ok=True)

    # ------------------------------------------------------------------
    # 公开接口
    # ------------------------------------------------------------------

    def export(self, ts_code: str, year: int, data: dict) -> Optional[Path]:
        """导出单份分析结果"""
        try:
            md_content = self._render_markdown(ts_code, year, data)
            path = self._note_path(ts_code, year, data)
```

```
            path.parent.mkdir(parents=True, exist_ok=True)
            path.write_text(md_content, encoding="utf-8")
            logger.info(f"[Obsidian] 导出: {path}")
            return path
        except Exception as e:
            logger.error(f"导出失败 {ts_code} {year}: {e}")
            return None

    def batch_export(self, results: dict[str, dict]) -> list[Path]:
        """批量导出，key格式为 'ts_code_year'，如 '600893.SH_2024'"""
        paths = []
        for key, data in results.items():
            try:
                ts_code, year_str = key.rsplit("_", 1)
                year = int(year_str)
                path = self.export(ts_code, year, data)
                if path:
                    paths.append(path)
            except Exception as e:
                logger.error(f"批量导出解析key失败 {key}: {e}")
        return paths

    # ------------------------------------------------------------------
    # 内部方法
    # ------------------------------------------------------------------

    def _note_path(self, ts_code: str, year: int, data: dict) -> Path:
        company = data.get("基本信息", {}).get("公司名称", ts_code)
        folder_name = f"{ts_code}-{company}"
        return self.vault_dir / "个股" / folder_name / f"{year}年报分析.md"

    def _render_markdown(self, ts_code: str, year: int, data: dict) -> str:
        info   = data.get("基本信息", {})
        fin    = data.get("核心财务数据", {})
        cf     = data.get("现金流质量", {})
        order  = data.get("订单与在手合同", {})
        val    = data.get("估值参考", {})

        company = info.get("公司名称", ts_code)
        today   = datetime.now().strftime("%Y-%m-%d")

        def fmt_item(d: dict, key: str) -> str:
            """格式化财务数据项"""
            v = d.get(key, {})
            if isinstance(v, dict):
                amt  = v.get("金额亿元", "N/A")
                yoy  = v.get("同比增长率", "")
```

```
                return f"{amt}亿" + (f" ({yoy}) " if yoy else "")
            return str(v) if v else "N/A"

        def bullet_list(items: list) -> str:
            if not items:
                return "- 暂无\n"
            return "".join(f"- {x}\n" for x in items)

        md = f"""---
tags: [财报, {ts_code}, {year}年报]
公司: {company}
股票代码: {ts_code}
报告期: {info.get('报告期', str(year))}
审计意见: {info.get('审计意见', '')}
分析日期: {today}
来源: Gemini自动分析
---

# {company} {year}年度报告分析

## 核心财务数据

| 指标 | 数值 |
|---|---|
| 营收 | {fmt_item(fin, '营收')} |
| 净利润 | {fmt_item(fin, '净利润')} |
| 扣非净利润 | {fmt_item(fin, '扣非净利润')} |
| 毛利率 | {fin.get('毛利率', 'N/A')} |
| 净利率 | {fin.get('净利率', 'N/A')} |
| ROE | {fin.get('ROE', 'N/A')} |
| 经营现金流 | {fmt_item(fin, '经营现金流')} |
| 合同负债 | {fmt_item(fin, '合同负债')} |
| 有息负债 | {fmt_item(fin, '有息负债')} |
| 资产负债率 | {fin.get('资产负债率', 'N/A')} |

## 现金流质量

- 经营现金流/净利润: **{cf.get('经营现金流_净利润比', 'N/A')}**
- 判断: {cf.get('判断', '')}

## 订单与合同

- 在手订单: {order.get('在手订单', 'N/A')}
- 新签合同: {order.get('新签合同', 'N/A')}
- 合同负债分析: {order.get('合同负债分析', '')}

## 关键变化
```

```
{bullet_list(data.get('关键变化', []))}

## 管理层核心表述

{bullet_list(data.get('管理层核心表述', []))}

## 风险因素

{bullet_list(data.get('风险因素', []))}

## 估值参考

| 指标 | 数值 |
|---|---|
| EPS | {val.get('EPS', 'N/A')} |
| PE区间 | {val.get('当前PE区间参考', '需结合市价计算')} |
| PB | {val.get('PB', 'N/A')} |

## Gemini结论

{data.get('分析师结论', '')}

## 值得深挖的问题

{bullet_list(data.get('值得深挖的问题', []))}

---

## 我的判断

> 此处手动填写

[[{ts_code}-投资论文]]
"""
        return md
```